

Inception — Eval Guide

Before the evaluation starts

The evaluator will run this command to wipe all Docker state. This is normal — your project must build from scratch cleanly.

```
docker stop $(docker ps -qa); docker rm $(docker ps -qa); \
docker rmi -f $(docker images -qa); \
docker volume rm $(docker volume ls -q); \
docker network rm $(docker network ls -q) 2>/dev/null
```

Preliminaries — Credentials check

What they check: No passwords or API keys appear in the git repository outside of properly configured secrets.

How to demonstrate:

```
git log --oneline          # show commit history is clean
cat .gitignore            # show srcs/.env and srcs/secrets/ are excluded
ls srcs/secrets/          # show the secrets directory exists with .txt files
cat srcs/.env             # show it contains NO passwords – only names/usernames/domain
```

What to say: “Passwords are stored in srcs/secrets/*.txt files which are gitignored. The .env file only contains non-sensitive configuration like domain name and usernames.”

General Instructions

1. Files are in a srcs folder at the root

```
ls -la                    # show Makefile at root, srcs/ at root
ls srcs/                  # show docker-compose.yml, .env, requirements/, secrets/
```

2. No network: host or links: in docker-compose.yml

```
grep -E "network: host|links:" srcs/docker-compose.yml || echo "Clean – none found"
```

3. Networks are present

```
grep "networks:" srcs/docker-compose.yml
```

4. No --link anywhere

```
grep -r "\-link" . || echo "Clean – none found"
```

5. No background processes or infinite loops in Dockerfiles/scripts

```
grep -rE "tail -f|sleep infinity|while true|& bash|nginx & |bash$" srcs/ || echo "Clean"
```

What to say: “All entrypoint scripts end with `exec <process>` which replaces the shell with the process as PID 1. There are no background processes, no infinite loops, no `tail -f` hacks.”

6. Containers built from penultimate Alpine/Debian

```
head -1 srcs/requirements/nginx/Dockerfile
head -1 srcs/requirements/wordpress/Dockerfile
head -1 srcs/requirements/mariadb/Dockerfile
```

All three show FROM `alpine:3.20` — a specific pinned version, not latest.

7. Run the Makefile

```
make
```

Activity Overview — Verbal Questions

“How do Docker and docker compose work?”

Docker builds isolated containers from Dockerfiles — each container is a lightweight process with its own filesystem and network namespace. `docker compose` is a tool that reads `docker-compose.yml` and manages multiple containers together: building their images, creating shared networks and volumes, and starting them in the right order with the right configuration.

“Difference between a Docker image used with docker compose vs without?”

Without docker compose you run each container manually with `docker run`, specifying every flag (ports, volumes, networks, env vars) by hand, one container at a time. With docker compose you declare everything in one YAML file and manage the whole stack with one command. The underlying Docker image is identical — docker compose is just orchestration on top.

“Benefit of Docker compared to VMs?”

VMs emulate an entire operating system including a full kernel, so they’re heavy (gigabytes, minutes to start). Docker containers share the host kernel and only package the application and its dependencies — they’re megabytes and start in seconds. You can run many more containers than VMs on the same hardware, and they’re reproducible: the same image runs identically everywhere.

“Pertinence of the directory structure?”

The structure matches what the subject requires: - Makefile at the root — single entry point - `srcs/` — all configuration files isolated here - `srcs/secrets/` — sensitive credentials stored separately, gitignored - `srcs/requirements/<service>/` — one Dockerfile per service, self-contained

README Check

```
head -1 README.md # must show the italicized first line
grep -E "## Description|## Instructions|## Resources|## Project" README.md
```

Simple Setup

Access via HTTPS only

```
# In browser: open https://msuokas.42.fr – WordPress site loads
# In browser: open http://msuokas.42.fr – connection refused (port 80 not exposed)
```

Verify TLS certificate

Click the padlock in the browser → “Certificate is not valid” (self-signed is expected). Check that it shows TLSv1.2 or TLSv1.3.

```
openssl s_client -connect msuokas.42.fr:443 2>/dev/null | grep -E "Protocol|Cipher"
```

Docker Basics

One Dockerfile per service, built from Alpine

```
ls srcs/requirements/nginx/
ls srcs/requirements/wordpress/
ls srcs/requirements/mariadb/
```

Images have same name as their service

```
docker images | grep -E "nginx|wordpress|mariadb"
# Expected: nginx:42, wordpress:42, mariadb:42
```

All containers running, no crashes

```
docker compose -f srcs/docker-compose.yml ps
```

All three show Up.

Docker Network

```
grep -A5 "networks:" srcs/docker-compose.yml
docker network ls | grep inception
docker network inspect inception
```

What to say: “Docker networks are virtual networks that allow containers to communicate with each other by container name as hostname. Containers on the same network can reach each other; containers NOT on the same network are isolated. Using a custom network (instead of network: host) means the containers are sandboxed — nothing from outside the network can reach MariaDB or WordPress directly. Only NGINX is exposed to the host via port 443.”

NGINX with SSL/TLS

```
docker compose -f srcs/docker-compose.yml ps nginx
curl -k https://msuokas.42.fr          # should return WordPress HTML
curl http://msuokas.42.fr              # should fail / connection refused
```

Demonstrate TLS version

```
openssl s_client -connect msuokas.42.fr:443 -tls1_2 2>/dev/null | grep "Cipher"
openssl s_client -connect msuokas.42.fr:443 -tls1_3 2>/dev/null | grep "Cipher"
```

What to say about the NGINX config: “NGINX listens only on port 443 with SSL. The config enforces `ssl_protocols TLSv1.2 TLSv1.3` — anything below that (TLS 1.0, 1.1, plain HTTP) is rejected. The SSL certificate is self-signed, generated at container startup with openssl, stored in `/etc/nginx/ssl/`. NGINX acts as the single entry point and proxies PHP requests to the WordPress container on port 9000 via FastCGI.”

WordPress with php-fpm and its volume

```
docker compose -f srcs/docker-compose.yml ps wordpress
docker volume ls | grep wordpress
docker volume inspect wordpress
# Check "device" shows /home/msuokas/data/wordpress
```

No nginx in the WordPress Dockerfile

```
grep -i nginx srcs/requirements/wordpress/Dockerfile || echo "Clean – no nginx"
```

Demonstrate the WordPress site works

1. Open `https://msuokas.42.fr` — site loads
2. Log in at `https://msuokas.42.fr/wp-admin`
 - Admin user: `forstman1` (does NOT contain “admin”)
 - Password: in `srcs/secrets/wp_admin_password.txt`
3. Add a comment as the non-admin user (`sami1`)
4. Edit a page in the dashboard and verify it updates on the frontend

Verify two users exist

```
docker exec -it mariadb mariadb -u ss -p lol1 \
-e "SELECT user_login, user_email FROM lol1.wp_users;"
```

MariaDB and its volume

```
docker compose -f srcs/docker-compose.yml ps mariadb
docker volume ls | grep mariadb
docker volume inspect mariadb
# Check "device" shows /home/msuokas/data/mariadb
```

No nginx in the MariaDB Dockerfile

```
grep -i nginx srcs/requirements/mariadb/Dockerfile || echo "Clean – no nginx"
```

Log into the database and show it is not empty

```
docker exec -it mariadb mariadb -u ss -p
# password is in srcs/secrets/db_password.txt
# then inside mariadb:
show databases;
```

```
use lol1;
show tables;
select count(*) from wp_posts;
```

What to say: “MariaDB is initialized with `mysql_install_db` on first run. The database, user, and permissions are set up using `mysqld --bootstrap` which reads SQL from stdin and exits — no background process needed. The `mysqld_safe` process is then launched with `exec` as PID 1.”

Persistence

1. Reboot the VM
2. After reboot:

```
make up
# or
docker compose -f srcs/docker-compose.yml up -d
```

3. Open `https://msuokas.42.fr` — site loads with all previous content intact

What to say: “Data persists because both WordPress and MariaDB use Docker volumes backed by bind mounts to `/home/msuokas/data/`. Even if containers are destroyed, the data on the host filesystem survives. The `firstmount` guard in each script also prevents WordPress from re-running the installer on restart.”

Configuration Modification

The evaluator will ask you to change a port. Example: change NGINX from 443 to 4343.

```
# In srcs/docker-compose.yml, under nginx:
ports:
  - "4343:443"
```

Then:

```
make re
# or
docker compose -f srcs/docker-compose.yml up -d --build
curl -k https://msuokas.42.fr:4343
```

Change it back after:

```
ports:
  - "443:443"
```

Useful diagnostic commands

```
# See all running containers
docker ps
```

```
# See logs for a specific container
docker logs nginx
docker logs wordpress
docker logs mariadb
```

```
# Get a shell inside a container
```

21.5.2026 klo 14.05

```
docker exec -it nginx sh
docker exec -it wordpress sh
docker exec -it mariadb sh
```

```
# Inspect a volume
docker volume inspect wordpress
docker volume inspect mariadb
```

```
# Check the docker network
docker network inspect inception
```

```
# Verify TLS
openssl s_client -connect msuokas.42.fr:443 2>/dev/null | head -20
```